ACOUSTIC CAMERA DIY PROJECT

DOA Radar System

Full Reference Manual

For the engineer who will read, edit, and extend this codebase.
Assumes knowledge of: **NumPy, Python, FreeRTOS threads**
No prior Qt or networking experience required.

Contents

1 The Big Picture — What This System Does . . . 3
2 Files Overview — The Role of Each File . . . 3
2.1 config.py — The Single Source of Truth . . . 4
3 Qt and QThreads — The FreeRTOS Analogy . . . 5
3.1 The Main Event Loop — Qt's Scheduler . . . 5
3.2 QThread — How to Read It . . . 5
3.3 Signals and Slots — The Inter-Thread Queue . . . 5
3.4 QMutex — The Critical Section . . . 6
4 UDP Deep Dive — From STM32 to Python . . . 7
4.1 UDP vs TCP — Why UDP . . . 7
4.2 The Socket — Every Line Explained . . . 7
4.3 The Receive Loop . . . 8
4.4 The Packet Format — Bytes to Channels . . . 8
4.4.1 Why Interleaved Format? . . . 8
4.4.2 The Unpack Steps . . . 9
5 DSP Worker — Signal Processing Explained . . . 10
5.1 The Thread Loop Pattern . . . 10
5.2 Step 1: Bandpass Filter . . . 10
5.3 Step 2: SRP-PHAT Algorithm . . . 11
5.3.1 The Core Intuition . . . 11
5.3.2 Pre-computed Delay Table . . . 11
5.3.3 The SRP-PHAT Loop . . . 11
6 UI Components — What You See on Screen . . . 13
6.1 Component Map . . . 13
6.2 MainWindow — The Coordinator . . . 13
6.3 ChannelCard — The Oscilloscope . . . 14
6.4 SpectrumPlot — The Beamformer View . . . 14

6.5 DOAIndicator — The Angle Readout 14
7 The Test Script — Simulating the STM32 15
7.1 How MockDataGenerator Works 15
7.2 How to Run It 15
8 Your Task Today: Channel Frontend Verification 16
8.1 Phase 1: Software Baseline (No Hardware Required) 16
8.2 Phase 2: Hardware Baseline — Silent Room Test 16
8.3 Phase 3: The Impulse Test (Finger Tap) 16
8.4 Phase 4: Channel Order Verification 17
8.5 Phase 5: Frequency Response Check 17
9 Known Issue: Spectrum Plot is Flat 18
9.1 Step 1: Modify `dsp_worker.py` 18
9.2 Step 2: Modify `main_window.py` 18
10 STM32 Integration Checklist 19
10.1 Pseudocode: FreeRTOS Audio Streaming Task 19
11 Quick Debugging Reference 20

1 The Big Picture — What This System Does

Before reading a single line of code, you need a mental model of the whole system. You have 8 microphones on a linear array. A sound source (a person speaking, a clap, a tone) is somewhere in front of the array. The question the software answers is: **from which angle is the sound coming?**

The answer is computed by measuring tiny **time differences** in when the sound arrives at each microphone. A sound from 45° left arrives at the leftmost mic a few microseconds before it arrives at the rightmost mic. The SRP-PHAT algorithm scans all possible angles from −90° to +90° and asks: "at which angle does the cross-correlation between all mic pairs peak?" That peak angle is the Direction of Arrival (DOA).

The software pipeline has exactly three stages, each running in its own thread:

```
+-----------------------------------------------------------+
|                   STM32 HARDWARE                          |
|  8 mics -> ADC -> interleaved int16 -> UDP packet -> LAN  |
+----------------------------+------------------------------+
                             |  UDP packet (raw bytes, 4096B)
                             v
+-----------------------------------------------------------+
|              UDPWorker  (Thread 1)                        |
|  Receives packets, parses bytes -> float32 NumPy array    |
|  Emits:  raw_packet  signal ->                            |
+----------------------------+------------------------------+
                             |  np.ndarray  shape (8, 256)
                             v
+-----------------------------------------------------------+
|              DSPWorker  (Thread 2)                        |
|  Bandpass filter  ->  SRP-PHAT  ->  peak angle (degrees)  |
|  Emits:  result  signal ->                                |
+----------------------------+------------------------------+
                             |  (waveform ndarray, float angle)
                             v
+-----------------------------------------------------------+
|              MainWindow  (UI Thread)                      |
|  Updates 8 channel cards, spectrum plot, DOA indicator    |
+-----------------------------------------------------------+
```

NOTE: This pipeline is deliberately **one-directional**. Data only flows downward. No thread ever reaches back up to pull data from the one above it. This is the same philosophy as FreeRTOS producer/consumer queues.

2 Files Overview — The Role of Each File

The project has 7 Python files. Here is what each one owns and does **not** touch:

File	Responsibility	Does NOT touch
`config.py`	Global constants only. No logic.	Nothing — pure data
`udp_worker.py`	Network socket, bytes to numpy array	DSP, UI, drawing

`dsp_worker.py`	Bandpass filter and SRP-PHAT math	Sockets, UI, drawing
`main_window.py`	Window layout, wiring threads together	Math, sockets
`plot_widgets.py`	Channel waveform cards and spectrum plot	Math, sockets, threads
`doa_indicator.py`	The big angle readout and slider widget	Everything else
`main.py`	Entry point — creates QApplication and shows window	Everything else
`test_gui_radar.py`	Simulates STM32 data for UI testing	Real hardware

2.1 config.py — The Single Source of Truth

This file is intentionally the **only** place where hardware parameters live. If you change the number of microphones, the sample rate, or the UDP port, you change it **here and nowhere else**. Every other file imports from it.

```
# The most important parameters for your hardware testing:

N_CHANNELS  = 8       # Must match the number of ADC channels on STM32
SAMPLE_RATE = 48000   # Must match STM32 ADC sampling frequency (Hz)
BLOCK_SIZE  = 256     # Samples per processing frame
MIC_SPACING = 0.05    # Physical distance between mics in meters (5 cm)
UDP_PORT    = 5005    # STM32 must send to this port number
BUFFER_SIZE = 4096    # 8 channels x 256 samples x 2 bytes = 4096 bytes exactly
```

WARNING: `BUFFER_SIZE` is **not** arbitrary. It is calculated as: `N_CHANNELS * BLOCK_SIZE * 2` (because `int16` = 2 bytes per sample). 8 x 256 x 2 = 4096. If your STM32 sends a different packet size, you must update both `BUFFER_SIZE` and `BLOCK_SIZE` together.

3 Qt and QThreads — The FreeRTOS Analogy

You already understand FreeRTOS tasks. Qt threads work the same way at a conceptual level. Let's map every concept you know to its Qt equivalent.

3.1 The Main Event Loop — Qt's Scheduler

In FreeRTOS, the scheduler runs your tasks. In Qt, the **event loop** does the same job. When you call `app.exec()` in `main.py`, you are starting the Qt scheduler. It runs forever, dispatching events (button clicks, timer ticks, incoming signals) to the correct handler functions.

```
FreeRTOS concept                    Qt (PyQt6) equivalent
____________________________        ____________________________________

vTaskStartScheduler()           ->  app.exec()
xTaskCreate(func, ...)          ->  QThread subclass with run() method
vTaskDelete(NULL)               ->  thread.quit()  then  thread.wait()
xQueueSend(queue, data, ...)    ->  signal.emit(data)
xQueueReceive(queue, buf)       ->  @pyqtSlot  decorated function
portDISABLE_INTERRUPTS()        ->  QMutex.lock()
portENABLE_INTERRUPTS()         ->  QMutex.unlock()
vTaskDelay(100/portTICK_MS)     ->  self.msleep(100)
```

3.2 QThread — How to Read It

Every thread in this project is a class that inherits from `QThread` and overrides the `run()` method. That `run()` method **is** the task function — it is what executes on the new thread's stack, just like your FreeRTOS task function pointer.

```python
class UDPWorker(QThread):
    # This is the "queue" that carries data to the next thread.
    # Declaring it here at class level is how Qt knows it is a signal.
    raw_packet = pyqtSignal(np.ndarray)

    def run(self):           # <- This is the FreeRTOS task function
        self._running = True
        while self._running: # <- The infinite task loop (like for(;;))
            data = socket.recvfrom(...)
            self.raw_packet.emit(data)  # <- xQueueSend equivalent
```

To **start** a thread (equivalent to `xTaskCreate`):

```python
self._udp.start()   # Calls run() on a brand new OS thread
```

To **stop** it cleanly:

```python
# In UDPWorker.stop():
self._running = False   # Signal the loop to exit
self.wait()             # Block until the thread actually finishes
                        # (like ulTaskNotifyTake or vTaskDelete + delay)
```

3.3 Signals and Slots — The Inter-Thread Queue

This is the most important Qt concept. In FreeRTOS you pass data between tasks using a queue. In Qt you use **signals** and **slots**.

A **signal** is declared as a class attribute using `pyqtSignal`:

```
raw_packet = pyqtSignal(np.ndarray)
```

This creates a typed "queue channel" that carries `np.ndarray` objects. To **send** data into it (from the producer thread):

```
self.raw_packet.emit(my_array)   # Non-blocking. Like xQueueSend with timeout=0.
```

On the consumer side, any regular Python method decorated with `@pyqtSlot` becomes a receiver. The **connection** is made in `setup_connections()`:

```
# In main_window.py setup_connections():
self._udp.raw_packet.connect(self._dsp.process)
```

This tells Qt: "whenever `raw_packet` fires on the UDP thread, call `self._dsp.process(data)` on the DSP thread." From this moment on, the wiring is automatic — you never call `process()` manually.

```
UDPWorker thread                      DSPWorker thread
------------------------              ------------------------------

raw_packet.emit(array)    ---------->  process(array)  <- called automatically
                                       by Qt's internal delivery mechanism
```

> **NOTE:** Qt's signal/slot mechanism is **thread-safe by default** when connecting across threads. Qt uses a hidden internal queue so the data is safely handed off between thread contexts. This is called a **Queued Connection** and is the automatic behavior when signal and slot live in different threads. You do not need to add your own mutex around `emit()`.

3.4 QMutex — The Critical Section

In `dsp_worker.py`, a `QMutex` protects `self._pending`. This is the shared variable where the UDP thread deposits new data and the DSP thread picks it up.

```
# Equivalent to  taskENTER_CRITICAL() / taskEXIT_CRITICAL()

# In process() -- called from UDP thread context via signal:
self._mutex.lock()
self._pending = data   # Write the new data
self._mutex.unlock()

# In run() -- on the DSP thread:
self._mutex.lock()
data          = self._pending   # Read and take ownership
self._pending = None            # Clear the slot
self._mutex.unlock()
```

Without this mutex, both threads could modify `self._pending` at the same instant, causing a race condition — exactly the same class of bug you would get in FreeRTOS if two tasks shared a global buffer without a semaphore.

4 UDP Deep Dive — From STM32 to Python

UDP is the transport protocol this system uses. Understanding it completely is critical for debugging hardware communication issues.

4.1 UDP vs TCP — Why UDP

TCP is like registered mail: every packet is acknowledged, lost packets are re-sent, and data arrives in order. For audio, this is catastrophic — if a packet from 10 ms ago is delayed, you do not want to wait for it. You want the **current** data now, even if that means skipping stale data.

UDP is like a radio broadcast: the sender fires packets and never checks whether they arrived. If a packet is lost, it is gone. For real-time audio streaming, this is exactly what you want.

```
TCP (not used here):
STM32 ->[send]->[wait for ACK]->[re-send if no ACK]-> PC
                ^
                This wait introduces latency and jitter. Unacceptable.

UDP (used here):
STM32 ->[send]->[send]->[send]->[send]->[send]-> PC
        Fire and forget. Maximum speed. No handshaking.
```

4.2 The Socket — Every Line Explained

```python
# 1. Create a UDP socket
#    AF_INET  = use IPv4 addresses (192.168.x.x format)
#    SOCK_DGRAM = "datagram" mode = UDP
#    (SOCK_STREAM would be TCP — we do NOT want that)
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 2. Increase the OS receive buffer to 64 KB
#    The OS kernel holds incoming packets here while Python is busy processing.
#    Think of it as your FreeRTOS queue length x item size.
#    Default is around 8 KB. At 48kHz with 8 channels, we fill it fast.
sock.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 65536)

# 3. Bind to address
#    "0.0.0.0" = listen on ALL network interfaces simultaneously
#    (Ethernet, WiFi, USB-RNDIS — all of them at once)
#    UDP_PORT = 5005 — must match exactly what your STM32 sends to
sock.bind(("0.0.0.0", 5005))

# 4. Set receive timeout to 1 second
#    Without this, recvfrom() blocks FOREVER when no data arrives.
#    With a 1 second timeout, the thread wakes up periodically.
#    This is how the STOP button works: it sets _running=False,
#    and within at most 1 second the thread checks the flag and exits.
sock.settimeout(1.0)
```

4.3 The Receive Loop

```
while self._running:
    try:
        # recvfrom() blocks here until a packet arrives OR timeout fires
        data, addr = sock.recvfrom(BUFFER_SIZE)
        # 'data' is now a Python bytes object: b'\x00\x1a\xff\x3c...'
        # 'addr' is the sender's (IP_string, port_int) tuple — not used here

        # --- parse and emit (see next section) ---

    except socket.timeout:
        continue   # Perfectly normal. Just loop and check _running again.

    except Exception as e:
        self.error.emit(str(e))   # Real error: signal the UI and exit
        break

sock.close()   # Always clean up the socket when done
```

TIP: To test UDP manually **without** the STM32, run this Python script on the same PC. Open a second terminal and run it while `main.py` is running:

```
import socket, numpy as np, time

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
while True:
    # 8 channels x 256 samples = 2048 int16 values = 4096 bytes
    fake = np.random.randint(-3000, 3000,
                             size=(256, 8), dtype=np.int16)
    sock.sendto(fake.tobytes(), ("127.0.0.1", 5005))
    time.sleep(0.005)   # ~200 packets per second
```

All 8 channels should show random noise in the UI. This verifies the entire software pipeline works before touching any hardware.

4.4 The Packet Format — Bytes to Channels

This is the most critical data transformation. Debugging almost every hardware integration problem starts here.

4.4.1 Why Interleaved Format?

The STM32 ADC with DMA in multi-channel scan mode naturally produces interleaved output. It samples all channels in a round-robin sequence, so in memory the output looks like:

```
Time flowing right ->
ADC samples: [CH0][CH1][CH2][CH3][CH4][CH5][CH6][CH7][CH0][CH1][CH2]...
             |--- sample 0 (S0) ----------------------|--- sample 1...

In the byte buffer (each [] = 2 bytes = one int16):
+----+----+----+----+----+----+----+----+----+----+----+----+
| C0 | C1 | C2 | C3 | C4 | C5 | C6 | C7 | C0 | C1 | C2 |..|
| S0 | S0 | S0 | S0 | S0 | S0 | S0 | S0 | S1 | S1 | S1 |..|
+----+----+----+----+----+----+----+----+----+----+----+----+
```

```
  Where S = sample index (0..255),  C = channel index (0..7)

  Total: 256 samples x 8 channels x 2 bytes = 4096 bytes per packet
```

4.4.2 The Unpack Steps

```python
# What arrives:
data = b'\x00\x10\xff\x20\x00\x30...'   # 4096 raw bytes from recvfrom()

# Step 1: Interpret every 2 bytes as one signed 16-bit integer
arr = np.frombuffer(data, dtype=np.int16)
# arr.shape = (2048,)   <- flat 1D array of 2048 integers
# Values range: -32768 to +32767  (full ADC range)

# Step 2: Normalize to float32 in range [-1.0, +1.0]
arr = arr.astype(np.float32) / 32768.0
# arr.shape still = (2048,)
# Now values range: -1.0 to +1.0

# Step 3: Reshape to (n_samples, n_channels)
arr = arr.reshape(-1, 8)
# arr.shape = (256, 8)
# arr[sample_index, channel_index]
# Row 0: [S0_C0, S0_C1, S0_C2, S0_C3, S0_C4, S0_C5, S0_C6, S0_C7]
# Row 1: [S1_C0, S1_C1, ...]

# Step 4: Transpose to (n_channels, n_samples)
arr = arr.T
# arr.shape = (8, 256)
# arr[channel_index, sample_index]
# Row 0 (arr[0]): all 256 samples of Channel 0  <- mic 0's waveform
# Row 1 (arr[1]): all 256 samples of Channel 1  <- mic 1's waveform
# ...
# Row 7 (arr[7]): all 256 samples of Channel 7  <- mic 7's waveform
```

After `arr.T`, each row is a complete time-domain waveform from one microphone. This is exactly what the `ChannelCard` oscilloscope displays and exactly what `DSPWorker` processes.

WARNING: If your STM32 firmware sends channels in a different order (for example, the physical connector wiring has CH7 on pin 0), every channel in the UI will be misidentified. The DOA algorithm will compute a wrong angle. To fix in software without changing firmware: `arr = arr[[7, 6, 5, 4, 3, 2, 1, 0], :]` to fully reverse, or `arr = arr[[0, 2, 1, 3, 4, 5, 6, 7], :]` to swap specific channels.

5 DSP Worker — Signal Processing Explained

The `DSPWorker` does two things: clean the audio with a bandpass filter, then find the direction with the SRP-PHAT algorithm.

5.1 The Thread Loop Pattern

The DSP thread uses a polling loop with `_pending` as the shared buffer. The signal from the UDP thread just **deposits** data into `_pending`, and the DSP thread picks it up whenever it finishes the previous frame.

```
def run(self):
    self._running = True
    while self._running:

        # Critical section: safely grab whatever the UDP thread deposited
        self._mutex.lock()
        data          = self._pending   # Take it
        self._pending = None            # Clear the slot for next delivery
        self._mutex.unlock()

        if data is not None:
            # Do the heavy math
            filtered = sosfilt(self._sos, data, axis=1).astype(np.float32)
            angle    = self._srp_phat(filtered)
            # Send result to the UI thread
            self.result.emit(data, float(angle))
        else:
            self.msleep(5)   # Nothing to do. Sleep 5 ms. Like vTaskDelay(5).
```

Important implication: if the UDP thread sends data faster than the DSP thread can process it, frames are **dropped**. The new deposit overwrites `_pending` before DSP reads it. This is intentional — for real-time display you always want the newest data, not a backlog of old frames.

5.2 Step 1: Bandpass Filter

The Butterworth bandpass filter removes everything outside 300 Hz – 3400 Hz. This eliminates DC offset (ruins cross-correlation), high-frequency ADC noise, and low-frequency mechanical vibration from the table or enclosure.

```
# Computed once in __init__() — expensive to compute, cheap to apply:
self._sos = butter(
    4,              # Filter order. Higher = sharper rolloff. 4 is a good trade-off.
    [300, 3400],    # [low_cutoff, high_cutoff] in Hz
    btype='bandpass',
    fs=48000,       # Must match SAMPLE_RATE in config.py
    output='sos'    # Second-Order Sections = numerically stable. Always use this.
)

# Applied to every incoming frame:
filtered = sosfilt(self._sos, data, axis=1)
# axis=1 means: filter along the samples dimension (axis 1)
#               independently for each channel (axis 0)
```

```
# data.shape    = (8, 256)
# filtered.shape = (8, 256)   <- same shape, but cleaned
```

5.3 Step 2: SRP-PHAT Algorithm

SRP-PHAT (Steered Response Power with PHAse Transform) is the core direction- finding algorithm. Here is the intuition first, then the code.

5.3.1 The Core Intuition

If a sound comes from angle theta, the same wavefront hits mic i and mic j at slightly different times. The time difference (TDOA) is:

```
TDOA(i, j, theta) = (mic_pos[i] - mic_pos[j]) * sin(theta) / speed_of_sound

In samples:  tau(i, j, theta) = TDOA * sample_rate
```

If we look at the cross-correlation between mic i and mic j, it will have a peak at exactly `tau`. The SRP algorithm **steers** to a candidate angle theta, looks up the cross-correlation at the expected delay, and accumulates power across all 28 microphone pairs (from 8 mics: 8 choose 2 = 28 pairs). The candidate angle where the total accumulated power is highest is the DOA.

PHAT (PHAse Transform) normalizes the cross-spectrum by its magnitude before taking the IFFT. This makes the cross-correlation sharper and more robust to colored noise.

5.3.2 Pre-computed Delay Table

All integer sample delays are computed **once** in `__init__` and stored:

```python
def _precompute_delays(self):
    angles_rad = np.deg2rad(SCAN_ANGLES)
    # SCAN_ANGLES = [-90, -89, ..., 0, ..., 89, 90]
    # angles_rad.shape = (181,)

    mic_pos = np.arange(N_CHANNELS) * MIC_SPACING
    # With 8 mics at 5cm spacing:
    # mic_pos = [0.0, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.35]  (meters)

    for i in range(N_CHANNELS):
        for j in range(i+1, N_CHANNELS):  # Only upper triangle: 28 pairs total
            # Path difference in meters for each scan angle
            d = (mic_pos[i] - mic_pos[j]) * np.sin(angles_rad)  # shape (181,)
            # Convert to integer sample delays
            self._delays[(i, j)] = (d / SPEED_OF_SOUND * SAMPLE_RATE).astype(int)
            # self._delays[(0,1)] is shape (181,): one integer delay per angle
```

This is the "pre-compute once, use many times" pattern — the same optimization you would do in FreeRTOS by computing a lookup table in an init task.

5.3.3 The SRP-PHAT Loop

```python
def _srp_phat(self, data):
    n_fft = data.shape[1]           # = 256
    P = np.zeros(len(SCAN_ANGLES))  # Power accumulator: one value per angle

    for i in range(N_CHANNELS):
        for j in range(i+1, N_CHANNELS):  # 28 iterations total
```

```python
        # FFT of each microphone signal
        Xi = np.fft.rfft(data[i], n=n_fft)   # shape (129,) complex
        Xj = np.fft.rfft(data[j], n=n_fft)   # shape (129,) complex

        # Cross-spectrum: captures phase relationship between the two mics
        GCC = Xi * np.conj(Xj)   # complex multiplication

        # PHAT weighting: divide by magnitude -> keep only the phase angle
        # The +1e-10 prevents division by zero for silent frames
        PHAT = GCC / (np.abs(GCC) + 1e-10)

        # IFFT -> cross-correlation in time domain
        # A peak at index tau means mic i leads mic j by tau samples
        gcc_t = np.real(np.fft.irfft(PHAT, n=n_fft))  # shape (256,)

        # For each candidate angle, look up the cross-correlation
        # at the expected delay and add to the power accumulator
        delays = self._delays[(i, j)]  # shape (181,)
        for a_idx, tau in enumerate(delays):
            P[a_idx] += gcc_t[int(tau) % n_fft]
            # % n_fft handles negative delays by wrapping around the array

    # The angle with the highest accumulated power is the estimated DOA
    return float(SCAN_ANGLES[int(np.argmax(P))])
```

```
Final output: a single float, e.g. -27.0
Meaning: the dominant sound source is 27 degrees to the LEFT.
Positive = right side, Negative = left side.
```

6 UI Components — What You See on Screen

The UI has four visual components. Each is a separate class with a single clear responsibility.

6.1 Component Map

```
+-----------------------------------------------------------------+
|  MainWindow  (main_window.py)                                   |
|                                                                 |
|  +--------------------+  +-------------------------------------+  |
|  | CHANNEL MONITORING |  | SpectrumPlot  (plot_widgets.py)     |  |
|  |                    |  |  SRP-PHAT power vs. angle curve     |  |
|  | ChannelCard [0]    |  |  Dashed vertical line at DOA peak   |  |
|  | ChannelCard [1]    |  +-------------------------------------+  |
|  | ChannelCard [2]    |                                          |
|  | ChannelCard [3]    |  +-------------------------------------+  |
|  | ChannelCard [4]    |  | DOAIndicator  (doa_indicator.py)    |  |
|  | ChannelCard [5]    |  |  Big numeric angle display          |  |
|  | ChannelCard [6]    |  |  Horizontal slider position         |  |
|  | ChannelCard [7]    |  +-------------------------------------+  |
|  +--------------------+                                          |
+-----------------------------------------------------------------+
```

6.2 MainWindow — The Coordinator

`MainWindow` is not primarily a visual component — it is the **orchestrator**. It creates all threads, creates all widgets, and wires everything together in `setup_connections()`.

```python
def setup_connections(self):
    # The three-stage pipeline:
    self._udp.raw_packet.connect(self._dsp.process)    # Stage 1 -> Stage 2
    self._dsp.result.connect(self._on_result)           # Stage 2 -> Stage 3

    # Button actions:
    self.btn_connect.clicked.connect(self._on_connect)
    self.btn_stop.clicked.connect(self._on_stop)

    # Error handling:
    self._udp.error.connect(self._on_udp_error)
```

When `_on_result` fires (delivered safely from DSP thread to UI thread by Qt):

```python
@pyqtSlot(np.ndarray, float)
def _on_result(self, waveform: np.ndarray, angle: float):
    # waveform.shape = (8, 256) — the filtered multi-channel audio
    # angle = e.g. -27.0 — estimated DOA in degrees

    for i in range(N_CHANNELS):
        self.channel_cards[i].update_data(waveform[i])  # waveform[i].shape=(256,)

    self.spectrum_plot.update_spectrum(SCAN_ANGLES, np.zeros(...), angle)
    self.doa_indicator.set_angle(angle)
```

6.3 ChannelCard — The Oscilloscope

One `ChannelCard` per microphone. Each shows a live waveform and the peak amplitude value. `update_data()` is called every time a new frame arrives:

```
def update_data(self, data: np.ndarray):
    # data.shape = (256,) — one channel's 256 samples
    self.val_label.setText(f"{np.max(np.abs(data)):.2f}")
    self.curve.setData(data)   # pyqtgraph re-draws the waveform instantly
```

The `val_label` (the large number in each card) is your **primary diagnostic tool** during hardware testing. Silent room -> near `0.00`. Finger tap -> near `0.80` or higher.

6.4 SpectrumPlot — The Beamformer View

This widget shows the SRP-PHAT power spectrum: X axis = angle (-90 to +90), Y axis = normalized acoustic power at each candidate angle. A sharp narrow peak means a clear unambiguous direction. A flat curve means no clear source.

A vertical dashed line marks the detected peak angle.

> **NOTE:** Currently in the live code, `spectrum` is always `np.zeros(...)` — the spectrum curve shows a flat line at zero. The DOA angle is computed correctly, but the **power spectrum visualization** is not connected yet. The DSP worker needs to also return the `P` array. See Section 8 for the exact code change.

6.5 DOAIndicator — The Angle Readout

The large widget at the bottom right. Shows the estimated angle as a big number and moves a horizontal slider. Color changes based on a simplified "confidence" metric: cyan for |angle| < 45°, yellow for more extreme angles.

```
def set_angle(self, angle: float):
    self.doa_value.setText(f"{angle:.1f}")
    # Map from [-90, 90] degrees to slider range [0, 180]
    slider_val = int(np.interp(angle, [-90, 90], [0, 180]))
    self.doa_slider.setValue(slider_val)
```

7 The Test Script — Simulating the STM32

`test_gui_radar.py` is your most useful tool today. It bypasses the UDP socket entirely and feeds synthetic multi-channel audio directly into the UI pipeline. This lets you verify every UI component works before connecting any hardware.

7.1 How MockDataGenerator Works

```
class MockDataGenerator(QObject):
    raw_packet = pyqtSignal(np.ndarray)  # Same signal type as UDPWorker

    def generate(self):
        # Simulated sound source sweeps from -60 to +60 degrees and back
        self.target_angle += self._angle_step
        if abs(self.target_angle) > 60:
            self._angle_step *= -1

        t = np.linspace(0, 0.01, BLOCK_SIZE)
        # delay_offset introduces a phase difference between channels
        # that is proportional to the simulated angle
        delay_offset = np.sin(np.deg2rad(self.target_angle)) * 5

        data = np.zeros((N_CHANNELS, BLOCK_SIZE), dtype=np.float32)
        for i in range(N_CHANNELS):
            # Each channel gets the same 1 kHz sine, but with a
            # channel-dependent phase shift (simulating the propagation delay)
            sig   = 0.3 * np.sin(2*np.pi*1000*t + self.phase + (i * delay_offset))
            noise = 0.05 * np.random.normal(size=BLOCK_SIZE)
            data[i] = sig + noise

        self.raw_packet.emit(data)   # Inject directly, bypassing UDP socket
```

The `i * delay_offset` term is the key: it gives each channel a slightly different phase, mimicking what the real microphone array would see from a source at `target_angle`. The SRP-PHAT algorithm should detect this and output an angle close to `target_angle`.

7.2 How to Run It

```
python test_gui_radar.py
```

Expected result: the DOA indicator sweeps smoothly between approximately −60° and +60°. All 8 channel cards show animated sine-wave waveforms.

8 Your Task Today: Channel Frontend Verification

Your task is to verify that each of the 8 microphone frontend channels is working correctly. Use this exact procedure in order.

8.1 Phase 1: Software Baseline (No Hardware Required)

```python
python test_gui_radar.py
```

Must pass before proceeding:

- All 8 channel cards show animated waveforms
- DOA indicator sweeps smoothly back and forth
- No crash, no error dialog, no frozen UI

If this fails, the problem is in your Python environment (missing packages, wrong PyQt6 version). Fix this first.

8.2 Phase 2: Hardware Baseline — Silent Room Test

Connect the STM32, start streaming, run `python main.py`, click CONNECT.

In a completely silent room, observe the VAL label on each channel card:

Observation	Expected VAL	If wrong — root cause
All 8 channels	`0.00` to `0.04`	Any higher: check ground and power supply noise
One channel only is high	`> 0.15`	Loose mic connection or bad solder joint on that mic
All channels high	`> 0.15`	ADC reference voltage problem or PSU noise coupling
Some channels flatline at exactly 0.00	`0.00` always	DMA not sampling that channel — firmware issue

NOTE: "VAL" is `np.max(np.abs(data))` — the peak amplitude in the current frame. A healthy silent channel sits between 0.00 and 0.04 due to inherent ADC quantization noise. Higher values indicate real signal or interference.

8.3 Phase 3: The Impulse Test (Finger Tap)

Tap each microphone firmly **one at a time** with your fingertip. Watch the VAL labels on screen.

Expected result for a correct system:

```
Tap Mic 0 -> Channel 0 VAL spikes to ~0.7 or higher
             Channels 1..7 stay below 0.05
Tap Mic 1 -> Channel 1 VAL spikes to ~0.7 or higher
             Channels 0, 2..7 stay below 0.05
...and so on for each microphone.
```

Failure modes:

Symptom	Root Cause

Tapping mic N also spikes mic N+1	ADC crosstalk. Add decoupling caps. Increase analog signal path impedance
Tapping mic 2 spikes Channel 5 in UI	Channel wiring is swapped. Fix in firmware or add index remap in `udp_worker.py`
One channel never spikes no matter what	That mic or preamp is dead — check solder joints and VDD
All channels spike together from any single tap	Mechanical coupling — mics too rigidly attached to same board
VAL is always 0.00 even with loud sound	ADC not running — check STM32 DMA trigger and clock configuration

8.4 Phase 4: Channel Order Verification

The DOA algorithm assumes `data[0]` is the **leftmost** physical microphone and `data[7]` is the **rightmost**. If the order is wrong, the DOA angle will be mirrored or randomized.

Procedure:

1. Stand 1 meter to the physical **left** of the array (approximately −60°)
2. Clap or speak continuously toward the array
3. The DOA indicator should read approximately **negative** values (e.g. −55° to −65°)
4. Move to the physical **right** side
5. DOA indicator should read **positive** values (e.g. +55° to +65°)

If moving left gives a positive reading, your channel order is reversed. Fix it in `udp_worker.py` after the transpose:

```
arr = arr.T         # existing line
arr = arr[::-1, :] # add this line to flip all channels
```

8.5 Phase 5: Frequency Response Check

The bandpass filter in `dsp_worker.py` passes only 300 Hz – 3400 Hz.

Test:

Signal	Expected VAL	If wrong
100 Hz tone (loud)	`< 0.05`	Analog high-pass filter on PCB missing or wrong cutoff
1000 Hz tone	`> 0.3`	Software filter not running — check `dsp_worker.py` init
5000 Hz tone (loud)	`< 0.05`	ADC anti-aliasing filter missing or wrong cutoff

9 Known Issue: Spectrum Plot is Flat

Currently the SRP-PHAT power spectrum (the curve in `SpectrumPlot`) always shows a flat line because `_on_result` passes `np.zeros(...)` as the spectrum data. The DOA angle is correct, but the visualization is incomplete.

Here is the exact change needed to fix it:

9.1 Step 1: Modify `dsp_worker.py`

Change `_srp_phat` to return both the angle and the power array:

```
# Old return:
return float(SCAN_ANGLES[int(np.argmax(P))])

# New return:
peak_idx = int(np.argmax(P))
P_norm = P / (np.max(P) + 1e-6)   # Normalize to [0, 1]
return float(SCAN_ANGLES[peak_idx]), P_norm
```

Change the signal declaration to carry the spectrum array:

```
# Old:
result = pyqtSignal(np.ndarray, float)

# New:
result = pyqtSignal(np.ndarray, float, np.ndarray)
```

Update `run()` to emit the spectrum:

```
# Old:
angle = self._srp_phat(filtered)
self.result.emit(data, float(angle))

# New:
angle, spectrum = self._srp_phat(filtered)
self.result.emit(data, float(angle), spectrum)
```

9.2 Step 2: Modify `main_window.py`

Update the slot signature and the spectrum update call:

```
# Old:
@pyqtSlot(np.ndarray, float)
def _on_result(self, waveform: np.ndarray, angle: float):
    ...
    self.spectrum_plot.update_spectrum(SCAN_ANGLES, np.zeros(len(SCAN_ANGLES)), angle)

# New:
@pyqtSlot(np.ndarray, float, np.ndarray)
def _on_result(self, waveform: np.ndarray, angle: float, spectrum: np.ndarray):
    ...
    self.spectrum_plot.update_spectrum(SCAN_ANGLES, spectrum, angle)
```

10 STM32 Integration Checklist

Everything your STM32 firmware must implement to work with this software:

num	Requirement	Details
1	ADC sample rate = 48000 Hz	`SAMPLE_RATE` in config.py must match exactly
2	8 channels sampled simultaneously	Use DMA with 8-channel scan mode
3	Data format: int16, signed, interleaved	`[CH0,CH1,...,CH7, CH0,CH1,...]` repeated 256 times
4	Packet size = 4096 bytes exactly	8 ch x 256 samples x 2 bytes per sample
5	Send via UDP to PC port 5005	Destination IP = PC's IP on the same network
6	Send rate: 187 packets per second	One packet per 256-sample window. 48000/256 = 187.5
7	UDP only — no TCP connection setup	No handshaking, no ACK. Just sendto() in a loop.
8	Channels ordered left to right	CH0 = leftmost physical mic, CH7 = rightmost

10.1 Pseudocode: FreeRTOS Audio Streaming Task

```
// This is the FreeRTOS task structure your STM32 firmware should have:

#define N_CHANNELS   8
#define BLOCK_SIZE   256
#define PACKET_SIZE  (N_CHANNELS * BLOCK_SIZE * sizeof(int16_t))  // = 4096

// DMA fills this buffer in interleaved format automatically:
// [S0_C0][S0_C1]...[S0_C7][S1_C0][S1_C1]...[S255_C7]
int16_t adc_dma_buffer[BLOCK_SIZE][N_CHANNELS];

void AudioStreamTask(void *pvParameters) {
    uint8_t udp_buffer[PACKET_SIZE];

    for (;;) {
        // Wait for DMA to signal that one full block is ready
        // (DMA half-transfer or transfer-complete interrupt sets this)
        xSemaphoreTake(dma_block_ready, portMAX_DELAY);

        // Copy to UDP buffer (already in interleaved format — no reshaping needed)
        memcpy(udp_buffer, (uint8_t*)adc_dma_buffer, PACKET_SIZE);

        // Send via UDP (using LwIP or similar)
        udp_sendto(pcb, pbuf, &pc_ip_addr, PC_UDP_PORT);
    }
}
```

The DMA peripheral in multi-channel scan mode fills `adc_dma_buffer` naturally in interleaved order — no software re-ordering is needed on the STM32 side. The Python software does the de-interleaving with `reshape` and `.T`.

11 Quick Debugging Reference

Symptom	Likely file	What to check or do
App crashes on start	`main.py`	Run: `pip install PyQt6 pyqtgraph scipy`
CONNECT does nothing, no error	`udp_worker.py`	Port in use? Run: `netstat -an | grep 5005`
UDP Error dialog appears	`udp_worker.py`	STM32 not sending, or sending to wrong IP
All channels show 0.00 always	`udp_worker.py`	Packet size mismatch — verify `BUFFER_SIZE = 4096`
Channels 0 and 1 show same waveform	`udp_worker.py`	N_CHANNELS wrong in config. Should be 8.
DOA always reads 0.0	`dsp_worker.py`	All channels identical — no phase difference in data
DOA is always wrong direction	`config.py`	Check `MIC_SPACING` value or flip channel order
Spectrum plot is always flat	`main_window.py`	Known issue — see Section 8 for the fix
UI is laggy or frame rate is low	`dsp_worker.py`	SRP-PHAT too slow. Reduce SCAN_ANGLES step size from 1 to 2
Packets/s in status bar is 0	`udp_worker.py`	No UDP packets arriving. Check network and firewall

END OF REFERENCE MANUAL

Acoustic DOA Radar System — Channel Verification Edition